

DATA CLEANING REPORT

E-Commerce Sales Data Cleaning Project

AUTHOR	Chandrachud Sahi
DATASET	E-Commerce Sales Data (synthetic, generated with ChatGPT)
CLEANING ENVIRONMENT	SQLite Online

Executive Summary

SUMMARY

The cleaned dataset (700 rows, 12 columns) is ready to support core reporting use cases, including revenue and order-value reporting, category- and city-level sales analysis, and customer segmentation. Rows carrying a NULL in *quantity*, *unit_price*, or *payment_method* should be filtered or flagged in aggregate calculations until those fields are resolved, and time-series analysis will need *order_date* parsed into a single date type first (see Section 6.6 and the MySQL snippet below).

01 — Project Overview

This report documents a structured SQL-based data cleaning workflow applied to a synthetic ecommerce sales dataset. The raw data was generated using ChatGPT and deliberately seeded with the categories of data quality issues encountered routinely in production environments: missing values, duplicate records, inconsistent text formatting, logically invalid numeric entries, and statistical outliers. The methodology followed a consistent three-step procedure for each issue class: detection via a targeted diagnostic query, remediation via a defined and auditable rule, and documentation of the rationale behind each corrective decision, ensuring the process is fully reproducible by any analyst working from the same source file.

The complete workflow — from ingestion of the raw CSV into SQLite Online through to export of the final validated table — was executed within a single working session. Both the original (uncleaned) and final (cleaned) CSV files accompany this report, enabling the transformation to be independently verified on a row-by-row basis against the documented procedure below.

Rationale: Data quality directly determines the reliability of any downstream analytics, reporting, or business decision. Data cleaning is not a cosmetic preprocessing step; it is the step that determines whether derived metrics can be trusted.

02 — What Is Data Cleaning?

Data cleaning, also referred to as data cleansing or data scrubbing, is the systematic process of identifying and correcting — or removing — data that is inaccurate, incomplete, inconsistently formatted, or logically invalid. Data sourced from forms, sensors, manual entry, or programmatic generation rarely arrives in a state suitable for direct analysis; it typically contains gaps, formatting irregularities, duplicate records, and values that violate basic business logic. Left unaddressed, these defects can propagate into downstream calculations, distorting aggregate metrics and reducing confidence in any conclusion drawn from the data.

This project follows the standard six-stage data cleaning methodology used across the analytics industry. Each stage in this report is explicitly labeled with its corresponding industry term, allowing the workflow to be mapped directly onto any standard data quality framework:

- **Data Profiling / Baseline Assessment** — establish dataset structure and characteristics prior to modification
- **Handling Missing Data** — identify null and blank values, then apply a defined treatment per column
- **Data Standardization** — normalize casing, whitespace, and spelling variants to a single canonical value
- **Data Validation** — identify entries that are technically present but violate business logic constraints
- **Outlier Detection & Treatment** — identify statistically extreme values and confirm their validity

- **Deduplication** — remove exact-match duplicate records while retaining canonical entries

Decision Framework: NULL vs. Drop vs. Impute

Because every remediation choice in this project follows one of three treatments, the rule used to select between them is stated once here and applied consistently throughout:

- **Set to NULL** — used when a value is missing or logically invalid (e.g. blank *payment_method*, negative *unit_price*) and no reliable evidence exists elsewhere in the row to reconstruct it. NULL preserves the record and marks the field as "unresolved" rather than fabricating a number.
- **Impute (rule-based)** — used only when a deterministic relationship exists elsewhere in the same row, as with *state*, which was derived from the already-standardized *city* value. This is not statistical estimation; it is a lookup against a fixed, verifiable mapping.
- **Drop the row** — reserved for exact full-row duplicates only (Stage 5). No row was dropped for a missing or invalid value, since doing so would discard otherwise-valid information in the remaining 11 columns.
- **Flag / leave as a documented limitation** — used when the environment lacks the capability to resolve the issue reliably, as with the three mixed *order_date* formats in SQLite.

03 — Tools and Environment

Three tools were used throughout this engagement:

METRIC	RESULT
ChatGPT	Generated the synthetic raw e-commerce sales dataset used in this project.
SQLite Online	A free, browser-based SQL tool used to import the CSV and run every query.
SQL (SQLite dialect)	The query language used to inspect, diagnose, and fix every issue.

04 — Dataset Overview & Baseline

INDUSTRY STAGE *Data Profiling / Baseline Assessment*

Prior to any modification, the raw CSV was imported into SQLite Online and inspected to confirm column names and data types. The table comprises 12 columns describing individual ecommerce orders: *order_id*, *order_date*, *customer_id*, *customer_name*, *city*, *state*, *category*, *product*, *quantity*, *unit_price*, *payment_method*, and *order_status*.

Two findings emerged from initial inspection. First, multiple columns exhibited missing values, duplicate records, and inconsistent text formatting. Second, *order_date* was stored as plain TEXT rather than a native DATE type — a constraint accounted for in subsequent stages.

Counting the starting rows

QUERY

```
SELECT COUNT(*)
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** This query establishes the row-count baseline for the entire engagement. It functions as

a control total: every subsequent stage that removes or nullifies records can be measured against this figure, providing an auditable basis for the before-and-after reconciliation presented in the final results summary.

05 — Stage 1: Handling Missing Values

INDUSTRY STAGE *Handling Missing Data*

This stage systematically evaluated every column for null and blank values. Each diagnostic query compares the total row count against the count of non-null values in the target column, and additionally tests for empty-string entries using TRIM, since a blank value stored as an empty string is not captured by a standard NULL check. This column-by-column methodology ensures comprehensive coverage regardless of how the gap was originally encoded.

Wherever a blank string is converted to NULL in this stage, the same rationale applies throughout: a single, consistent encoding for "missing" simplifies downstream filtering, aggregation, and join logic — a query only needs to check *IS NULL* once, rather than testing for every possible blank-string variant a column might contain.

5.1 — order_id, order_date, customer_id, customer_name

QUERY

```
SELECT COUNT(*) - COUNT(order_id) AS missing_order_id
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** Subtracting the non-null count from the total row count yields the missing-value count for the column in a single pass, avoiding the need for a WHERE clause. `order_id` functions as the primary key for this dataset, so its integrity was verified first, since a missing identifier would compromise traceability for any downstream join or aggregation.

QUERY

```
SELECT COUNT(*) AS missing_dates
FROM ecommerce_sales
WHERE order_date IS NULL OR TRIM(order_date) = '';
```

→ **WHAT THIS DOES:** The same diagnostic logic was applied to `order_date`, extended to test for both true NULL values and empty-string entries, since `order_date` is a TEXT-typed column where a blank string would not be captured by a NULL-only check.

QUERY

```
SELECT COUNT(*) AS missing_customer_id
FROM ecommerce_sales
WHERE customer_id IS NULL OR TRIM(customer_id) = '';
```

→ **WHAT THIS DOES:** This check validates referential completeness for `customer_id`. A missing identifier in this column would sever the relationship between an order and its originating customer, which would affect

any customer-level segmentation, lifetime-value calculation, or retention analysis performed downstream.

QUERY

```
SELECT COUNT(*) AS missing_customer_name
FROM ecommerce_sales
WHERE customer_name IS NULL OR TRIM(customer_name) = '';
```

→ **WHAT THIS DOES:** The identical NULL-or-blank diagnostic was applied to `customer_name` to confirm completeness of the customer attribution field.

5.2 — city and state

QUERY

```
SELECT COUNT(*) AS missing_city
FROM ecommerce_sales
WHERE city IS NULL OR TRIM(city) = '';
```

→ **WHAT THIS DOES:** The city column was evaluated using the same NULL-or-blank diagnostic pattern to confirm geographic completeness prior to downstream standardization in Stage 2.

QUERY

```
SELECT COUNT(*) AS missing_state
FROM ecommerce_sales
WHERE state IS NULL OR TRIM(state) = '';
```

→ **WHAT THIS DOES:** The identical diagnostic was applied to the state column to determine completeness of the geographic hierarchy at the state level.

QUERY

```
SELECT *
FROM ecommerce_sales
WHERE state IS NULL OR TRIM(state) = '';
```

→ **WHAT THIS DOES:** Prior to determining a remediation strategy, the affected rows were retrieved directly for inspection. This step assessed whether a deterministic relationship existed between the city and state columns that could later support a confident, rule-based imputation rather than an arbitrary default value.

Decision Rationale: Rather than imputing a state value immediately, blank entries were converted to true NULL. This preserves data integrity and defers resolution until a reliable inference method becomes available — specifically, the city column once standardized.

QUERY

```
UPDATE ecommerce_sales
SET state = NULL
WHERE state IS NOT NULL AND TRIM(state) = '';
```

→ **WHAT THIS DOES:** Empty-string entries that were not technically NULL were normalized to true NULL values, ensuring all missing entries in this column are represented through a single, consistent encoding for downstream filtering, aggregation, and imputation logic.

5.3 — category and product

QUERY

```
SELECT COUNT(*) AS missing_category
FROM ecommerce_sales
WHERE category IS NULL OR TRIM(category) = '';
```

→ **WHAT THIS DOES:** The category column was evaluated using the established NULL-or-blank diagnostic pattern, consistent with the methodology applied to prior columns.

QUERY

```
SELECT COUNT(*) AS missing_product
FROM ecommerce_sales
WHERE product IS NULL OR TRIM(product) = '';
```

→ **WHAT THIS DOES:** The product column was evaluated for missing values using the standard NULL-or-blank diagnostic.

QUERY

```
UPDATE ecommerce_sales
SET product = NULL
WHERE product IS NOT NULL AND TRIM(product) = '';
```

→ **WHAT THIS DOES:** Consistent with the remediation rule applied to state, blank entries were normalized to NULL rather than substituted with an inferred product name, preserving the principle that no value should be fabricated in the absence of supporting evidence.

5.4 — quantity, unit_price, payment_method, order_status

QUERY

```
SELECT COUNT(*) - COUNT(quantity) AS missing_quantity
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** Missing-value detection for quantity used the count-difference method rather than a

TRIM-based check, since this is a numeric column where empty-string entries are not a valid data type and therefore not a relevant failure mode.

QUERY

```
SELECT COUNT(*) - COUNT(unit_price) AS missing_unit_price
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** The same count-difference diagnostic was applied to `unit_price`, consistent with its status as a numeric column.

QUERY

```
SELECT COUNT(*) AS missing_payment_method
FROM ecommerce_sales
WHERE payment_method IS NULL OR TRIM(payment_method) = '';
```

→ **WHAT THIS DOES:** The `payment_method` column was evaluated for missing values using the NULL-or-blank diagnostic. This check surfaced the largest data quality gap identified across the dataset.

Business impact. At 143 rows (roughly 20% of the dataset), missing `payment_method` reduces the reliability of payment-mix reporting, fraud/chargeback analysis, and channel-level reconciliation with payment processor statements, and can understate counts for whichever method is under-represented in the remaining data. Three remediation paths are available going forward, in order of preference: (1) **look up the true value from transaction or gateway logs** where `order_id` can be joined against processor records — the most accurate option where feasible; (2) **label the gap explicitly as "Unknown"** rather than leaving it NULL, so payment-mix reports can show a labeled "unattributed" bucket rather than dropping rows without explanation; or (3) **targeted imputation** using the customer's most frequent historical payment method, used only where logs are unavailable and the imputation is clearly flagged as modeled rather than observed data.

QUERY

```
UPDATE ecommerce_sales
SET payment_method = NULL
WHERE payment_method IS NOT NULL AND TRIM(payment_method) = '';
```

→ **WHAT THIS DOES:** All blank `payment_method` entries were normalized to NULL, maintaining consistency with the missing-value treatment policy applied throughout this stage.

QUERY

```
SELECT COUNT(*) AS missing_order_status
FROM ecommerce_sales
WHERE order_status IS NULL OR TRIM(order_status) = '';
```

→ **WHAT THIS DOES:** The `order_status` column, the final field in the missing-value audit, was evaluated using the standard NULL-or-blank diagnostic to complete column-by-column coverage of the schema.

06 — Stage 2: Fixing Text Inconsistencies

INDUSTRY STAGE *Data Standardization*

Text columns are particularly susceptible to inconsistency, since a single real-world value can be represented through multiple encodings — extraneous whitespace, inconsistent capitalization, or alternate spellings. Left uncorrected, a SQL GROUP BY operation would treat 'Mumbai', 'mumbai', and 'bombay' as three distinct entities rather than one. Each text column was inspected using a DISTINCT query, and every inconsistency identified was resolved using TRIM, LOWER, and targeted UPDATE statements that map each variant to a single canonical value.

6.1 — customer_name

QUERY

```
SELECT DISTINCT customer_name
FROM ecommerce_sales
ORDER BY customer_name;
```

→ **WHAT THIS DOES:** A DISTINCT query enumerated every unique value in customer_name to perform a manual inspection for residual whitespace, inconsistent capitalization, or spelling variation prior to standardization.

QUERY

```
UPDATE ecommerce_sales
SET customer_name = TRIM(customer_name)
WHERE customer_name IS NOT NULL;
```

→ **WHAT THIS DOES:** Leading and trailing whitespace was removed from every value using TRIM, ensuring that logically identical customer names resolve to a single, exact string match during grouping or joins.

6.2 — city

QUERY

```
SELECT DISTINCT city
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** A DISTINCT query enumerated every unique city value to identify whitespace irregularities, inconsistent casing, and spelling variants requiring standardization.

QUERY


```

UPDATE ecommerce_sales SET city = LOWER(TRIM(city)) WHERE city IS NOT NULL;
UPDATE ecommerce_sales SET city = 'New Delhi'
  WHERE city = 'new delhi' OR city = 'delhi';
UPDATE ecommerce_sales SET city = 'Bengaluru'
  WHERE city = 'bangalore' OR city = 'bengaluru';
UPDATE ecommerce_sales SET city = 'Mumbai'
  WHERE city = 'mumbai' OR city = 'bombay';
UPDATE ecommerce_sales SET city = 'Pune'
  WHERE city = 'pune' OR city = 'poona';
UPDATE ecommerce_sales SET city = 'Lucknow'
  WHERE city = 'lucknow';

```

→ **WHAT THIS DOES:** The remediation followed a two-phase standardization pattern: first, every value was normalized to a consistent baseline using TRIM and LOWER to eliminate superficial formatting variance; second, each known spelling variant was explicitly mapped to a single, correctly capitalized canonical city name via targeted UPDATE statements. This normalize-then-map approach is a standard pattern for resolving categorical inconsistency at scale.

6.3 — state

QUERY

```

SELECT DISTINCT state
FROM ecommerce_sales
WHERE state IS NOT NULL;

```

→ **WHAT THIS DOES:** A DISTINCT query enumerated unique state values to identify the same categories of whitespace, casing, and spelling inconsistency previously observed in the city column.

QUERY

```

UPDATE ecommerce_sales SET state = LOWER(TRIM(state)) WHERE state IS NOT NULL;
UPDATE ecommerce_sales SET state = 'Delhi'
  WHERE state = 'nct delhi' OR state = 'delhi';
UPDATE ecommerce_sales SET state = 'Karnataka'
  WHERE state = 'karnatak' OR state = 'karnataka';
UPDATE ecommerce_sales SET state = 'Maharashtra'
  WHERE state = 'maharashtra' OR state = 'maharastra';
UPDATE ecommerce_sales SET state = 'Uttar Pradesh'
  WHERE state = 'uttarpradesh' OR state = 'up' OR state = 'uttar pradesh';

```

→ **WHAT THIS DOES:** The same normalize-then-map methodology applied to city was used here, ensuring every known typographical error or abbreviation resolves to a single, correctly formatted state name.

6.4 — category

QUERY

```
SELECT DISTINCT category
FROM ecommerce_sales
WHERE category IS NOT NULL;
```

→ **WHAT THIS DOES:** A DISTINCT query enumerated unique category values to identify formatting inconsistencies and spelling errors requiring correction.

QUERY

```
UPDATE ecommerce_sales SET category = LOWER(TRIM(category)) WHERE category IS NOT NULL;
UPDATE ecommerce_sales SET category = 'Fashion'
  WHERE category = 'fashion' OR category = 'fashon';
UPDATE ecommerce_sales SET category = 'Home'
  WHERE category = 'home' OR category = 'hme';
UPDATE ecommerce_sales SET category = 'Electronics'
  WHERE category = 'electronics' OR category = 'electronic' OR category = 'electornics';
```

→ **WHAT THIS DOES:** The same normalize-then-map pattern was applied, consolidating misspelled variants such as 'fashon' and 'electornics' into their correct, single canonical category name.

6.5 — product, payment_method, order_status

QUERY

```
SELECT DISTINCT product
FROM ecommerce_sales
WHERE product IS NOT NULL;
```

→ **WHAT THIS DOES:** The product column was inspected for formatting inconsistency. Unlike city or category, this field exhibited only whitespace irregularities, with no spelling variants requiring categorical mapping.

QUERY

```
UPDATE ecommerce_sales
SET product = TRIM(product)
WHERE product IS NOT NULL;
```

→ **WHAT THIS DOES:** Extraneous whitespace was removed via TRIM. No further variant-mapping was required, as no spelling inconsistencies were present in this column.

QUERY

```
SELECT DISTINCT payment_method
FROM ecommerce_sales
WHERE payment_method IS NOT NULL;
```

→ **WHAT THIS DOES:** The payment_method column was inspected for whitespace and formatting inconsistency following the same DISTINCT-query methodology applied to other text columns.

QUERY

```
UPDATE ecommerce_sales
SET payment_method = TRIM(payment_method)
WHERE payment_method IS NOT NULL;
```

→ **WHAT THIS DOES:** Extraneous whitespace was removed from every payment_method value via TRIM.

QUERY

```
SELECT DISTINCT order_status
FROM ecommerce_sales
WHERE order_status IS NOT NULL;
```

→ **WHAT THIS DOES:** The order_status column was inspected for whitespace and formatting inconsistency using the standard DISTINCT-query methodology.

QUERY

```
UPDATE ecommerce_sales
SET order_status = TRIM(order_status)
WHERE order_status IS NOT NULL;
```

→ **WHAT THIS DOES:** Extraneous whitespace was removed from every order_status value via TRIM, completing the text standardization pass across all categorical columns.

6.6 — order_date format inconsistency

QUERY

```
SELECT DISTINCT order_date
FROM ecommerce_sales
ORDER BY order_date;
```

→ **WHAT THIS DOES:** A DISTINCT query enumerated unique order_date values to assess format consistency across the column prior to any temporal analysis.

Documented Limitation: SQLite has no native equivalent to MySQL's STR_TO_DATE() for parsing textual month representations, so the three formats could not be consolidated into a single DATE-typed column within this environment. This constraint is documented rather than resolved through an unreliable manual workaround; remediation is deferred to a subsequent pass using pandas or a SQL engine with more flexible date parsing capabilities.

Recommended next step. The snippet below shows how the three mixed formats can be normalized into a single parsed_date column once the table is migrated to (or mirrored in) MySQL, which — unlike SQLite —

supports STR_TO_DATE() with explicit format masks:

MYSQL — NORMALIZE THREE DATE FORMATS

```
UPDATE ecommerce_sales
SET parsed_date = CASE
  WHEN order_date REGEXP '[0-9]{4}-[0-9]{2}-[0-9]{2}$'
  THEN STR_TO_DATE(order_date, '%Y-%m-%d')
  WHEN order_date REGEXP '[0-9]{2}/[0-9]{2}/[0-9]{4}$'
  THEN STR_TO_DATE(order_date, '%m/%d/%Y')
  WHEN order_date REGEXP '[0-9]{2}-[A-Za-z]+-[0-9]{4}$'
  THEN STR_TO_DATE(order_date, '%d-%M-%Y')
  ELSE NULL
END;
```

The CASE expression routes each row to the correct format mask based on a regex match against its shape, writing the result into a new native DATE column (parsed_date) and leaving any row that matches none of the three known patterns as NULL for manual review, rather than misparsing it without a record of the failure.

07 — Stage 3: Cleaning Invalid Values

INDUSTRY STAGE *Data Validation*

Certain values pass a NULL check without issue yet remain logically invalid — for example, a negative order quantity or a non-positive unit price. These entries are easily overlooked because they are technically present, but they can still affect any downstream calculation that depends on them, such as a revenue total or average order value. This stage applied range-based validity constraints to surface this category of hidden data quality issues.

7.1 — Numeric validity checks

QUERY

```
SELECT *
FROM ecommerce_sales
WHERE quantity <= 0;
```

→ **WHAT THIS DOES:** This query identifies records where quantity violates the basic business constraint that a placed order must involve a strictly positive unit count; zero or negative values are logically impossible for a completed transaction.

QUERY

```
UPDATE ecommerce_sales
SET quantity = NULL
WHERE quantity <= 0;
```

→ **WHAT THIS DOES:** In the absence of a defined business rule or supporting metadata to infer the correct quantity, affected values were set to NULL rather than substituted with an estimated figure, preserving the principle that no value should be fabricated without justification.

QUERY

```
SELECT *
FROM ecommerce_sales
WHERE unit_price <= 0;
```

→ **WHAT THIS DOES:** The same validity constraint was applied to unit_price, on the basis that a legitimate product transaction cannot carry a price of zero or less.

QUERY

```
UPDATE ecommerce_sales
SET unit_price = NULL
WHERE unit_price <= 0;
```

→ **WHAT THIS DOES:** Logically invalid prices were set to NULL, consistent with the preserve-rather-than-estimate policy applied throughout this engagement.

7.2 — Categorical validity checks

QUERY

```
SELECT DISTINCT payment_method
FROM ecommerce_sales
ORDER BY payment_method;
```

→ **WHAT THIS DOES:** Every unique payment_method value was reviewed against the expected set of valid payment categories to identify any unrecognized or out-of-domain entries that would indicate a data entry or system integration error.

QUERY

```
SELECT DISTINCT order_status
FROM ecommerce_sales
ORDER BY order_status;
```

→ **WHAT THIS DOES:** The same domain-validity check was applied to order_status to confirm all values conform to the expected category set.

QUERY

```
SELECT DISTINCT category
FROM ecommerce_sales
ORDER BY category;
```

→ **WHAT THIS DOES:** The same domain-validity check was applied to category, now meaningful given that the column had already been standardized to canonical values during Stage 2.

QUERY

```
SELECT DISTINCT city
FROM ecommerce_sales
ORDER BY city;
```

→ **WHAT THIS DOES:** The same domain-validity check was applied to city to confirm all values conform to the expected geographic reference set.

7.3 — Resolving state using city as a reference

QUERY

```
SELECT DISTINCT state
FROM ecommerce_sales
ORDER BY state;

SELECT city, state
FROM ecommerce_sales
WHERE state IS NULL;
```

→ **WHAT THIS DOES:** This step revisited the 34 NULL state values deferred from Stage 1. Because every city in this dataset maps deterministically to exactly one state, the now-standardized city value in each row could be used as a reliable basis for imputation — a remediation path that was not viable until the city column itself had been cleaned and validated.

QUERY

```
UPDATE ecommerce_sales SET state = 'Delhi'
  WHERE city = 'New Delhi' AND state IS NULL;
UPDATE ecommerce_sales SET state = 'Maharashtra'
  WHERE (city = 'Mumbai' OR city = 'Pune') AND state IS NULL;
UPDATE ecommerce_sales SET state = 'Karnataka'
  WHERE city = 'Bengaluru' AND state IS NULL;
UPDATE ecommerce_sales SET state = 'Uttar Pradesh'
  WHERE city = 'Lucknow' AND state IS NULL;
```

→ **WHAT THIS DOES:** Missing state values were imputed by mapping each row's city to its corresponding state, converting a missing-data condition into a high-confidence, rule-based correction grounded in a fixed and verifiable geographic relationship rather than statistical estimation.

QUERY

```
SELECT DISTINCT product
FROM ecommerce_sales
ORDER BY product;
```

→ **WHAT THIS DOES:** This is the final domain-validity check in this stage, confirming that the product column contains no invalid or unexpected values following the standardization applied in Stage 2.

08 — Stage 4: Removing Outliers

INDUSTRY STAGE *Outlier Detection & Treatment*

An outlier is a value that deviates so substantially from the normal distribution of a column that it is unlikely to represent a genuine observation. Rather than applying automatic removal, each candidate outlier identified in this stage was investigated to determine whether it reflected a legitimate, if unusually large, transaction or a data-entry error.

8.1 — quantity outliers

QUERY

```
SELECT MIN(quantity), MAX(quantity)
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** MIN and MAX aggregate functions were used to establish the observed range for quantity, providing a basis for identifying values that fall outside a statistically plausible range for this attribute.

QUERY

```
SELECT order_id, product, quantity
FROM ecommerce_sales
ORDER BY quantity DESC
LIMIT 10;
```

→ **WHAT THIS DOES:** The ten highest quantity values were retrieved for manual investigation to determine whether the recurring value of 999 represented a legitimate bulk transaction or a data-entry artifact.

QUERY

```
UPDATE ecommerce_sales
SET quantity = NULL
WHERE quantity = 999;
```

→ **WHAT THIS DOES:** Values identified as statistically implausible were set to NULL rather than replaced with

an estimated figure, so the unresolved condition is recorded clearly rather than masked by a fabricated value.

8.2 — unit_price outliers

QUERY

```
SELECT MIN(unit_price), MAX(unit_price)
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** MIN and MAX aggregate functions were used to establish the observed range for unit_price as a basis for outlier identification.

QUERY

```
SELECT order_id, product, unit_price
FROM ecommerce_sales
ORDER BY unit_price DESC
LIMIT 10;
```

→ **WHAT THIS DOES:** The highest-priced records were retrieved for manual investigation to assess whether the recurring value of 999,999 represented a legitimate premium product price or a data-entry artifact.

QUERY

```
UPDATE ecommerce_sales
SET unit_price = NULL
WHERE unit_price = 999999;
```

→ **WHAT THIS DOES:** Affected unit_price values were set to NULL, again prioritizing data integrity over the introduction of an unverified estimate.

Why 999 and 999,999 were treated as error sentinels, not statistical outliers. A purely statistical method — the IQR rule (flagging values beyond $1.5 \times \text{IQR}$ from the nearest quartile) or a z-score threshold (e.g. $|z| > 3$) — was considered for this stage, but was set aside as the primary detection method here. Both 999 and 999,999 recurred as the exact same value across multiple, unrelated products (three different products at quantity 999; four different products at price 999,999). A genuine distribution of large orders or premium prices would be unlikely to produce the identical figure across unrelated items; that pattern is more consistent with a sentinel or placeholder value used by the data-entry system than with organic variance. A rule-based check for "an exact value repeated across unrelated rows" was therefore used instead of a purely statistical cutoff, which could either miss a repeated sentinel that falls inside the statistical bound, or flag legitimate high-value transactions that a business context (e.g. a genuine bulk order) would explain. In a production setting, an IQR or z-score pass would still be a useful complementary check for outliers that aren't exact repeats.

09 — Stage 5: Removing Duplicate Records

INDUSTRY STAGE *Deduplication*

A duplicate record, in this context, is one in which every column holds a value identical to another row in the

table. Duplicate records inflate row counts, totals, and averages, making deduplication one of the most consequential steps prior to analysis. Detection was performed by grouping on all columns simultaneously; any group containing more than one matching row was flagged as a duplicate set.

QUERY

```
SELECT *, COUNT(*) AS duplicate_count
FROM ecommerce_sales
GROUP BY order_id, order_date, customer_id, customer_name,
        city, state, category, product, quantity,
        unit_price, payment_method, order_status
HAVING COUNT(*) > 1;
```

→ **WHAT THIS DOES:** This query groups every row by all 12 columns simultaneously and filters for groups with a count greater than one. Any such group indicates two or more records that are identical across every field — a true exact duplicate rather than a near-match or partial overlap.

QUERY

```
DELETE FROM ecommerce_sales
WHERE rowid NOT IN (
    SELECT MIN(rowid)
    FROM ecommerce_sales
    GROUP BY order_id, order_date, customer_id, customer_name,
            city, state, category, product, quantity,
            unit_price, payment_method, order_status
);
```

→ **WHAT THIS DOES:** Exactly one canonical record was retained from each duplicate group, identified by the lowest internal rowid, with all remaining redundant copies deleted. This ensures no unique data is lost during deduplication — only exact, redundant repeats are removed.

QUERY

```
SELECT COUNT(*)
FROM ecommerce_sales;
```

→ **WHAT THIS DOES:** The final row count was captured following completion of all cleaning stages, providing a direct, auditable comparison against the 720-row baseline established at the outset of this engagement.

10 — Changes by Stage at a Glance

The table below gives a single-view count of every record removed, changed, or converted, broken out by cleaning stage, for quick visibility before the full results summary.

STAGE	ACTION	ROWS AFFECTED
1 — Missing Values	Blanks converted to NULL (state, product, payment_method)	34 + 34 + 143 = 211
2 — Text Standardization	Text columns normalized (casing / whitespace / spelling)	All rows, 7 columns
3 — Data Validation	Invalid numeric values set to NULL (quantity $\leq$ 0, unit_price $\leq$ 0)	12 + 3 = 15
3 — Data Validation	state imputed from standardized city	34
4 — Outlier Treatment	Sentinel values set to NULL (quantity = 999, unit_price = 999,999)	3 + 4 = 7
5 — Deduplication	Exact-duplicate rows deleted	20

11 — Final Results Summary

The table below consolidates the row counts before and after cleaning, alongside every issue identified and resolved during this engagement.

METRIC	RESULT
Rows before cleaning	720
Rows after cleaning	700
Duplicate rows removed	20
Missing values — state	34 → resolved via city-to-state mapping
Missing values — product	34 → standardized to NULL
Missing values — payment_method	143 → standardized to NULL
Invalid quantity values corrected	12 (zero or negative)
Invalid unit_price values corrected	3 (negative)
Outlier quantities corrected	3 rows at value 999
Outlier unit_price values corrected	4 rows at value 999,999
Text columns standardized	customer_name, city, state, category, product, payment_method, order_status
Known unresolved issue	order_date in three inconsistent formats (documented limitation)

12 — Appendix: Example Before → After Text Corrections

Representative examples of raw variants and their standardized canonical values from Stage 2, shown for the three columns with spelling and casing corrections.

COLUMN	ORIGINAL VARIANT	STANDARDIZED VALUE
city	' bangalore'	Bengaluru
city	'BOMBAY'	Mumbai
city	'delhi '	New Delhi
city	'poona'	Pune
state	'karnatak'	Karnataka
state	'maharastra'	Maharashtra
state	'up' / 'uttarpradesh'	Uttar Pradesh
state	'nct delhi'	Delhi
category	'fashon'	Fashion
category	'hme'	Home
category	'electornics' / 'electronic'	Electronics

13 — Limitations and Notes

- `order_date` could not be standardized into a single DATE-typed column within SQLite, as the engine lacks native support for parsing textual month representations equivalent to MySQL's `STR_TO_DATE()`. This constraint is documented explicitly rather than addressed through an unreliable manual workaround; see Section 6.6 for the recommended MySQL migration path.
- Invalid and outlier values in `quantity`, `unit_price`, and `payment_method` were set to NULL rather than estimated or removed, preserving the integrity of each record without introducing fabricated data.
- As the dataset is synthetic, the data quality issues addressed in this project were deliberately introduced for demonstration purposes. The detection and remediation techniques applied, however, are directly transferable to production e-commerce data.

14 — Conclusion

This project executed a complete, methodical data cleaning workflow: baseline assessment, missing value remediation, text standardization, validity correction, outlier treatment, and deduplication — corresponding directly to the six-stage methodology applied across the analytics industry. Every corrective decision, including the deliberate choice to preserve unresolved values as NULL rather than impute estimates, was made in service of transparency, auditability, and data integrity. The dataset was reduced from 720 to 700 rows following exactduplicate removal, with every other identified issue either resolved directly or explicitly documented as a known limitation for subsequent remediation.

Both the original (uncleaned) and final (cleaned) CSV files accompany this report, enabling the complete transformation to be independently verified against the procedure documented above.